

Gestión de la entrada y salida de procesos en Java

UD 1. Programación multiproceso

Programació de Serveis
i Processos

CFGS Desenvolupament
d'Aplicacions Multiplataforma



IES Jaume II El Just
Tavernes de la Valldigna

Departament d'Informàtica
Curs 2022-23

Contenidos

1	Introducción	3
2	Gestión de la entrada y salida de procesos mediante Streams	3
2.1	Ejemplo 1. Mostrando la información por pantalla	4
2.2	Ejemplo 2. Redirección a ficheros	7
3	Redirección con ProcessBuilder	9
3.1	Ejemplo con <code>ProcessBuilder.Redirect.Inherit</code>	9
3.2	Ejemplo para redirigir la salida a un fichero	10
4	ProcessBuilder y tuberías: StartPipeline	12
4.1	Tuberías de ProcessBuilder	12

1 Introducción

Como ya vimos en el apartado inicial de esta unidad, la entrada y salida de los procesos puede utilizarse para compartir información entre ellos.

Debemos tener en cuenta, que cuando lanzamos un proceso desde nuestros programas, éstos procesos no tienen una terminal asociada sobre la que operar con la entrada y salida.

Por ejemplo, el siguiente código, intenta lanzar el comando `ps aux` mediante la clase `ProcessBuilder`:

```
ProcessBuilder pb = new ProcessBuilder("ps", "aux");  
Process p = pb.start();
```

Aunque compilaría y se ejecutaría sin problemas, no sería capaz de mostrar nada por pantalla.

En este apartado, vamos a ver cómo podemos acceder a esta información proporcionada por los procesos que lanzamos mediante el uso de `Streams` de entrada y salida asociada a éstos.

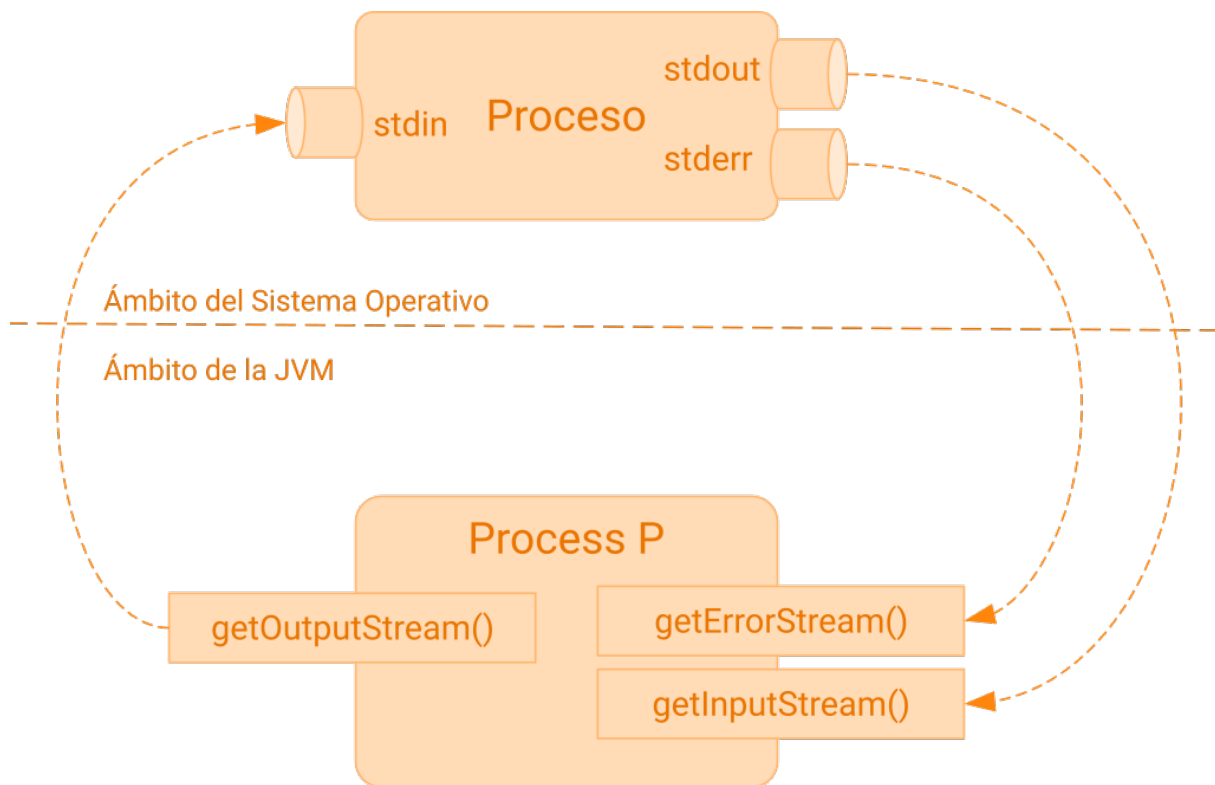
2 Gestión de la entrada y salida de procesos mediante Streams

Como comentamos, los procesos creados bien con `ProcessBuilder` o con `Runtime.exec` no disponen de una terminal (`tty`) asociada, por lo que no podemos trabajar directamente con las entradas y salidas estándar del proceso. Para ello, la clase `Process` nos ofrece varios *Streams* o flujos de bytes asociados la entrada y salida, y a los que podemos acceder mediante los siguientes métodos:

- `OutputStream getOutputStream()`: Devuelve el flujo de salida conectado a la entrada estándar del proceso.
- `InputStream getInputStream()`: Devuelve el flujo de entrada conectado a la salida estándar del proceso.
- `InputStream getErrorStream()`: Devuelve el flujo de entrada conectado a la salida de error del proceso.

Debemos tener presente que la clase `Process` representa un proceso, pero no es el proceso en sí. Con el uso de estos métodos, podemos proporcionar información al proceso a través del flujo de salida que tenemos conectada a él, y obtener información del proceso mediante los flujos de entrada.

Gráficamente, podríamos esquematizar esta relación del siguiente modo:



Además, cuando utilizamos estos tres métodos, deberemos tener en cuenta que los búffers de entrada y salida tienen una longitud limitada, por lo que si no se escriben y leen a continuación pueden llegar a bloquear el subprocesso.

2.1 Ejemplo 1. Mostrando la información por pantalla

Vamos a ver un ejemplo cómo trabajar con estos Streams de entrada y salida de los procesos en el siguiente programa `EjemploStreams1.java`. En él vemos cómo podemos obtener la salida estándar de un proceso lanzado por nosotros y volcarla en nuestra propia terminal:

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;

public class EjemploStreams1 {

    public static void main(String[] args) {

        System.out.println("----- Calendario del año 2022
        -----");
```

```
try {  
  
    // En primer lugar, creamos el objeto ProcessBuilder,  
    // y lo inicializamos con la orden que vayamos a utilizar  
    // (en nuestro caso, `cal 2022`)  
    ProcessBuilder pb = new ProcessBuilder("cal", "2022");  
  
    // Lanzamos el proceso con Start, que nos  
    // devuelve un objeto de tipo Process.  
    Process p = pb.start();  
  
    // Obtenemos la salida del proceso con getInputStream,  
    // Y la volcamos por pantalla, pasándola por un InputStreamReader  
    BufferedReader br = new BufferedReader(  
        new InputStreamReader(  
            p.getInputStream()));  
    String line;  
    while ((line = br.readLine()) != null) {  
        System.out.println(line);  
    }  
} catch (IOException e) {  
    e.printStackTrace();  
}  
}
```

En primer lugar, creamos la instancia de ProcessBuilder con el comando y ejecutamos éste mediante el método start, obteniendo el objeto Process correspondiente:

```
ProcessBuilder pb = new ProcessBuilder("cal", "2022");  
Process p = pb.start();
```

Una vez hemos lanzado el proceso, obtenemos el flujo de entrada conectado a la salida estándar del proceso:

```
InputStream is=p.getInputStream();
```

Para poder leer cómodamente de este Stream, el paquete `java.io` nos ofrece los decoradores `InputStreamReader` y `BufferedReader`, de modo que `InputStreamReader` recubre a `In-`

`putStream`, codificando los bytes en caracteres, y `BufferedReader` recubre a `InputStreamReader`, ofreciendo métodos para leer líneas directamente:

```
InputStreamReader isr=new InputStreamReader(is);  
BufferedReader br=new BufferedReader(isr);
```

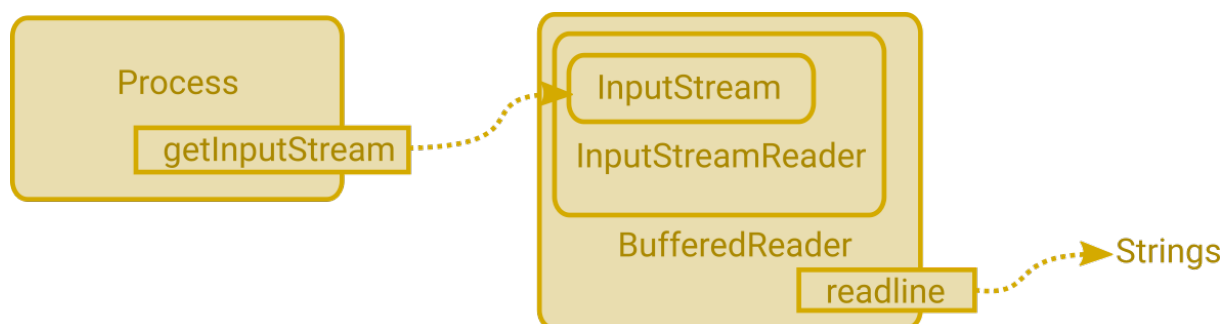
Aunque podemos definir directamente la variable `br` sin la necesidad de definir variables intermedias, del siguiente modo:

```
BufferedReader br = new BufferedReader(  
new InputStreamReader(  
    p.getInputStream()));
```

Con estos decoradores, podremos leer directamente líneas a partir de la salida estándar del proceso:

```
String line;  
while ((line = br.readLine()) != null) {  
    System.out.println(line);  
}
```

Gráficamente, podemos ver este proceso del siguiente modo:





Sobre la codificación de caracteres

Un aspecto a tener en cuenta cuando trabajamos con *streams* es la codificación de la información que se intercambia entre los procesos, y que depende del sistema operativo en que estemos trabajando. La mayoría de sistemas (GNU/Linux, Mac OS, Android, iOS..) utilizan la codificación *UTF-8*, basada en el estándar *Unicode*. Por su parte, MS Windows utiliza sus propios formatos, incompatibles con el resto, como Windows-1252. Así que para tratar los datos correctamente en Java cuando hacemos uso de mecanismos de comunicación entre procesos más avanzados, habrá que tener en cuenta el tipo de codificación que el propio sistema utiliza.

2.2 Ejemplo 2. Redirección a ficheros

En el siguiente programa `EjemploStreams2.java`, tenemos un ejemplo acerca de cómo redirigir esta salida hacia un fichero, utilizando para ello los decoradores `FileWriter` y `BufferedWriter`:

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;

public class EjemploStreams2 {

    public static void main(String[] args) {

        try {

            // En primer lugar, creamos el objeto ProcessBuilder,
            // y lo inicializamos con la orden que vayamos a utilizar
            // (en nuestro caso, `cal 2022`)
            ProcessBuilder pb = new ProcessBuilder("cal", "2022");

            // Lanzamos el proceso con Start, que nos
            // devuelve un objeto de tipo Process.
            Process p = pb.start();
```

```
// Obtenemos la salida del proceso con getInputStream,
// Y la volcamos por pantalla, pasándola por un InputStreamReader
BufferedReader br = new BufferedReader(
    new InputStreamReader(
        p.getInputStream()));

// Volcamos la salida a un fichero, utilizando el decodador
// BufferedWriter, de forma similar al ejemplo anterior
BufferedWriter bw = new BufferedWriter(
    new FileWriter(new File("calendario2022.txt")));

// Escribimos la cabecera
bw.write("----- Calendario del año 2022
-----");

// Leemos del stream y escribimos en el fichero
String line;
while ((line = br.readLine()) != null) {
    bw.write(line + "\n");
}
bw.close();

} catch (IOException e) {
    e.printStackTrace();
}

}
```

Como vemos, una vez tenemos preparado el `BufferedReader br` para la lectura del proceso, en lugar de mostrarlo por pantalla, lo volcamos al disco del siguiente modo:

```
BufferedWriter bw = new BufferedWriter(
    new FileWriter(
        new File("calendario2022.txt")));
String line;
while ((line = br.readLine()) != null) {
    bw.write(line + "\n");
}
bw.close();
```

Con ello generaremos el fichero `calendario2022.txt`, cuyo contenido será la salida que antes mostrábamos por pantalla.

3 Redirección con ProcessBuilder

La clase `ProcessBuilder` nos ofrece una forma más simple de redirigir la entrada y salida de un proceso, ya sea hacia la salida estándar, hacia un fichero o cualquier otra fuente o destino de datos, a través de métodos como `redirectOutput`, `redirectInput` o `redirectError`. Mediante estos métodos, únicamente deberemos indicar hacia dónde redirigir la salida.

Para indicar esta redirección, podemos utilizar la clase `ProcessBuilder.Redirect`, que puede tomar algunos de los siguientes valores:

- `ProcessBuilder.Redirect.PIPE`: Para indicar que el subprocesso de E/S se conectará al proceso Java actual a través de una tubería. Este es el comportamiento por defecto.
- `ProcessBuilder.Redirect.INHERIT`: Para indicar que el subprocesso de E/S tendrá los mismos flujos de E/S que el proceso actual.
- `ProcessBuilder.Redirect.DISCARD`: Descarta la salida del subprocesso.
- Una redirección para leer de un fichero, obtenida mediante `Redirect.from(fichero)`
- Una redirección para escribir en un fichero, obtenida mediante `Redirect.to(fichero)`
- Una redirección para escribir al final de un fichero, obtenida mediante el método `Redirect.appendTo(fichero)`

Veamos algunos ejemplos de ello.

3.1 Ejemplo con ProcessBuilder.Redirect.Inherit

En el siguiente ejemplo *EjemploRedirect1.java* se muestra la salida del proceso por la misma salida que el proceso Java, esto es, la pantalla:

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;

public class EjemploRedirect1 {
```

```
public static void main(String[] args) {

    System.out.println("----- Calendario del año 2022
    ↪ -----");
    try {

        // En primer lugar, creamos el objeto ProcessBuilder,
        // y lo inicializamos con la orden que vayamos a utilizar
        // (en nuestro caso, `cal 2022`)
        ProcessBuilder pb = new ProcessBuilder("cal", "2022");
        pb.redirectOutput(ProcessBuilder.Redirect.INHERIT);

        // Lanzamos el proceso con Start, que nos
        // devuelve un objeto de tipo Process.
        pb.start();

    } catch (IOException e) {
        e.printStackTrace();
    }

}
```

Como vemos, el programa define el `processBuilder`, redirige la salida como `INHERIT`, e inicia el proceso:

```
ProcessBuilder pb = new ProcessBuilder("cal", "2022");
pb.redirectOutput(ProcessBuilder.Redirect.INHERIT);
pb.start();
```

3.2 Ejemplo para redirigir la salida a un fichero

El siguiente programa *EjemploRedirect2.java* muestra cómo redirigir la salida a un fichero mediante `ProcessBuilder.Redirect.to`:

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileWriter;
```

```
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;

public class EjemploRedirect2 {

    public static void main(String[] args) {

        try {

            // En primer lugar, creamos el objeto ProcessBuilder,
            // y lo inicializamos con la orden que vayamos a utilizar
            // (en nuestro caso, `cal 2022`)
            ProcessBuilder pb = new ProcessBuilder("cal", "2022");

            File calendario=new File("calendario2022.txt");
            BufferedWriter bw = new BufferedWriter(
                new FileWriter(calendario));
            bw.write("----- Calendario del año 2022
-----\n");
            bw.close();

            // Para sobrescribir el fichero
            pb.redirectOutput(ProcessBuilder.Redirect.to(calendario));
            // Para ampliar el fichero
            //pb.redirectOutput(ProcessBuilder.Redirect.appendTo(calendario));

            // Lanzamos el proceso con Start, que nos
            // devuelve un objeto de tipo Process.
            pb.start();

        } catch (IOException e) {
            e.printStackTrace();
        }

    }
}
```

Tengamos en cuenta que con `Redirect.to`, vamos a sobrescribir el contenido del fichero. Si lo que deseamos es mantener el contenido, utilizaremos el método `Redirect.appendTo`. Por ejemplo:

```
ProcessBuilder pb = new ProcessBuilder("cal", "2022");
File calendario=new File("calendario2022.txt");
BufferedWriter bw = new BufferedWriter(new FileWriter(calendario));
bw.write("----- Calendario del año 2022 -----\\n");
bw.close();
pb.redirectOutput(ProcessBuilder.Redirect.appendTo(calendario));
```

4 ProcessBuilder y tuberías: StartPipeline

4.1 Tuberías de ProcessBuilder

La API de `ProcessBuilder` incorpora el método estático `startPipeline` que permite crear una tubería de procesos en los que la salida de un proceso se envía a la entrada del siguiente.

Este método se define del siguiente modo:

```
static List<Process> startPipeline(List<ProcessBuilder> builders)
```

Como vemos, este método recibe una lista de objetos de tipo `ProcessBuilder`, instanciados ya con sus propiedades correspondientes (comandos, entorno...), y devuelve una lista de objetos de tipo `Process`, como resultado de la puesta en marcha de cada proceso.

Así pues, si deseamos crear una tubería por ejemplo con la salida de:

```
ps aux | grep root
```

Definiríamos la lista de *builders* con los dos *ProcessBuilder*:

```
List builders = Arrays.asList(
    new ProcessBuilder("ps", "aux"),
    new ProcessBuilder("grep", "root"));
```

E invocáramos a `startPipeline` del siguiente modo:

```
List listaProcesos = ProcessBuilder.startPipeline(builders);
```

Dado que lo que nos interesa es la salida únicamente del último proceso ejecutado, obtendremos éste mediante:

```
Process ultimo = listaProcesos.get(listaProcesos.size() - 1);
```

Con lo que ya podremos utilizar los métodos `getInputStream` y `getErrorStream` para obtener la salida de todos ellos.